User manual

rcodeagent 0.1.0 | R package 0.1.0

José Gamarra-Moncayo

Assistance for programming, analysis and research within RStudio.

This manual accompanies the local 0.1.0 distribution. The chat runs in Viewer; the system browser is used for OAuth sign-in. Neither Codex CLI nor an API key is required. The assistant does not replace methodological review or code validation.

Reading guide: 1 Installation · 2 Account and interface · 3 Context and permissions · 4 Editing and execution · 5 Analysis and APA · 6 Continuity and export · 7 Troubleshooting · 8 Privacy and references.

Quick start: install the package, open Addins > rcodeagent, sign in, choose a model, capture context and begin in Normal. Enable Auto only when you want direct changes to the displayed target.

Maintainer contact: gamarramoncayoj@gmail.com

Project license: GPL (>= 3), GNU GPL version 3 or later. Full text is available in LICENSE.md.

1. Installation and first launch

Requirements

- R 4.2.0 or later and RStudio Desktop. Available native verification concerns Windows; identical behavior across platforms is not guaranteed.
- Internet for dependencies, sign-in and requests. Your account must have effective access to the service and compatible models.
- A writable user location for R packages, preferences, history and encrypted credentials.

Install the distribution

Close the addin and use Session > Restart R when replacing an earlier version. Run in the console:

```
install.packages("remotes") # first time only
remotes::install_local(file.choose(), upgrade = "never")
# Select rcodeagent_0.1.0.tar.gz in the file picker
rcodeagent::rcodeagent()
```

remotes resolves declared dependencies. If using the downloaded source directory, open that directory as the working directory and replace file.choose() with ".". When a dependency requires compilation, follow the build-tool requirements reported by R for your operating system.

Verify and launch

```
packageVersion("rcodeagent") # should report 0.1.0
rcodeagent::rcodeagent()
```

You can also use Addins > rcodeagent. Under Tools > Modify Keyboard Shortcuts, search for rcodeagent and assign an available shortcut. Launch from RStudio; Rscript and an external R console do not provide the editor context.

This distribution is prepared for GitHub, but the manual does not assume a remote release exists. Do not use an invented repository address or install.packages("rcodeagent") as though the package were available on CRAN.

If you used the former package, close it and restart R. Install and verify rcodeagent before removing the previous package through Packages to avoid duplicate Addins entries. Keep its user data: when no new profile exists, the old profile is reused for history, language, OAuth identity and encrypted credentials. If both profiles exist, the new one wins; profiles and secrets are not automatically merged or copied.

2. Account, model and interface

Connect your account

Model requests require an eligible ChatGPT Plus or Pro subscription under OpenAI's Sign in with ChatGPT quickstart. Free accounts are not supported for model usage in rcodeagent. Signing in alone does not grant permission to use models.

Click Sign in with ChatGPT in the sidebar. Complete authorization in the system browser, then return to RStudio. The callback uses a local loopback address. Connected indicates that the authorized catalog could be retrieved; saved credentials alone are not evidence of connectivity.

After disconnecting, use Sign in with ChatGPT to reconnect the last account, or Use another account to register a different one without the previous account's identity hints. Disconnecting the plugin does not sign out of ChatGPT in your browser; check the identity shown by OpenAI before authorizing. A cancelled sign-in notice closes after five seconds.

Never paste tokens or keys into the chat. The service determines available models. Disconnect attempts OpenAI session revocation and removes local tokens. If remote revocation is unconfirmed, it tells you: also disconnect the app in ChatGPT Settings. Close ends the assistant and retains credentials. An invalid refresh requires signing in again; temporary network errors preserve the encrypted session.

Find the controls

- Header: new conversation, history, show/hide sidebar and Close.
- Conversation: messages, response, waiting feedback and code proposals when applicable.
- Context: Current selection, Active file and Shared files, plus attachment chips.
- Composer: attachment clip, message, Auto, work-mode menu, model/effort and Send or Stop.
- Sidebar: account, editing target, export, About, language and status/usage.

Choose model and effort

Open the model name beside Send. Choose Model and Effort: Low, Medium, High or Very high; Medium is the initial value. Refresh models retrieves the account catalog. An unsupported effort produces an error rather than a silent downgrade. Response time and results depend on the model and service.

Enter submits; Shift+Enter adds a line. Stop terminates the pending request and invalidates late results. After a failed request, the draft can be restored without overwriting newer text.

English is the default interface language. Language switches the interface between English and Spanish, and your saved selection is preserved when reopening. It does not translate your messages or force a response language; state your preference in the request. About shows the author and installed package version. Spanish labels in this guide correspond to the same controls when Spanish is selected.

3. Share context and choose permissions

Targets and attachments serve different purposes

Current selection captures the chosen editor range. Active file captures the document and refreshes the active tab when submitting: a change revokes Auto. Choose .R script pins a target by editor ID. A captured selection also remains pinned until changed; it does not automatically follow your cursor.

An empty, unsaved source document is a valid target by ID, displayed as Unsaved document [ID]. The application does not infer an Untitled number or language from that label. Saved documents must be .R scripts for Auto. Always inspect the target label before editing.

Attachments are read-only context, not write authorization. Supported UTF-8 formats are .R, .md, .qmd and .Rmd, up to 512 KB per file, 2 MB total and 12 files. Shared files (Proyecto completo in Spanish) displays shared attachments; it does not scan the project or read every file. CSV, Excel and SPSS datasets are not accepted attachments in this version.

Work modes

- Normal reads authorized context, explains, plans, generates code and revises it only in chat. Ask for successive revisions and copy the result. Applying or running it in RStudio requires Auto or manual action.
- Auto requires explicit activation through the shield or menu. It edits only the selected target. Editing does not imply execution: code is submitted to the console only when explicitly requested.

Auto starts OFF in a new Job, new chat or history-resumed chat. Explicitly choosing another selection, active file or R script preserves Auto and cancels the previous request; Shared files and attachments do not change the editing target. Failed capture, incompatible targets, unrecaptured conflicts and IPC failures revoke Auto. Editor ID, path and full content are checked before effects. An empty unsaved R source buffer remains valid.

In Auto, an explicit execution request is submitted without another review or confirmation, including new code. "Run" uses the captured context; "Install package lavaan" submits install.packages("lavaan") without editing the script or automatically loading it. Questions, quotes, negations and conditional requests do not grant execution. File deletion, system commands and other identified administrative operations require reviewing the exact code and clicking Run this code; this confirmation is valid once. Each authorization covers one request and is never recovered from history or on Viewer return.

RStudio may queue submitted code until it becomes available. Chat Stop cancels pending model requests or preparation, not code already submitted. Use console controls for execution in progress. Submission is blocked if pending text is detected in the console input; this is not a busy-session detector and cannot eliminate races with subsequent input changes.

Execution has your R session permissions and can change objects and files; it is not a sandbox. Packages are not implicitly installed. There is no automatic Environment inspection or return of execution output to the model. Saving remains manual. Valid syntax does not guarantee security or statistical correctness.

4. Examples: discuss, edit and execute

A. Revise code within chat

Open a synthetic script, capture its context and leave Normal selected:

```
scores <- c(12, 15, NA, 18, 20)
average <- mean(scores)
```

Ask: "Explain why average returns NA and provide corrected code." Then: "Add the number of valid observations to your proposed code." Both replies remain in chat; the script stays unchanged. Copy the code or enable Auto to apply it.

B. Edit without executing

Check the target, enable Auto and ask: "Modify the calculation to omit missing values and add the number of valid observations. Preserve the rest." One reasonable result is:

```
scores <- c(12, 15, NA, 18, 20)
average <- mean(scores, na.rm = TRUE)
n_valid <- sum(!is.na(scores))
```

Auto edits the target without running or saving the code. Inspect the change. Select Normal or turn off the shield to return to chat-only work.

C. Load a library or run a range

With Auto and a selected target, "Load package lavaan" adds library(lavaan) if it is not already an independent call, then submits that call to the main console. The package must already be installed. "Write library(lavaan) in the script" edits only; "Run library(lavaan) without adding it to the script" executes only.

"Run line 32", "Run lines 32 to 40" and "Run the selection" use the captured target. Line numbers start at 1 in the full document. Empty or out-of-range lines and incomplete multiline expressions are rejected without silently expanding the range.

"Fix this block and run it" edits the target, verifies the result and submits the block without a second confirmation. Inspect Console, Plots or Viewer after submission: an acknowledgment does not prove successful completion. Errors are not automatically retried. In this example, verify average = 16.25 and n_valid = 4.

5. Examples: planning and an APA table

Plan the analysis before programming

In Normal: "Design an analysis plan for two paired measurements. Distinguish data quality, assumptions, primary analysis, effect size, confidence interval and reporting. Do not assume results." Share the relevant design, scale and data structure. The addin does not automatically know your objects or verify assumptions without evidence.

A reproducible synthetic example

Copy code from a Normal reply or ask Auto to insert it into the selected target:

```
pre  <- c(10, 12, 11, 13,  9, 14)
post <- c(12, 13, 11, 15, 10, 16)
pairs <- complete.cases(pre, post)
dif <- post[pairs] - pre[pairs]
result <- t.test(post[pairs], pre[pairs], paired = TRUE)
summary <- data.frame(
  n = sum(pairs),
  mean_pre = mean(pre[pairs]),
  mean_post = mean(post[pairs]),
  difference = mean(dif)
)
```

Suggested request: "Prepare R code for an APA 7 table using this example. Include n, means, standard deviations, the post-minus-pre difference, t, degrees of freedom, p and a confidence interval. Explain complete-pair handling and dependencies. Do not invent results or references."

The model may propose flextable or another dependency: review and explicitly install what is needed. Execute it manually or explicitly request a specific range in Auto. To obtain an interpretation, share relevant anonymized output as text or an accepted attachment; the assistant cannot read result from your Environment.

Check difference direction, denominators, rounding, labels, confidence interval, notes, italics and absence of vertical rules. This illustrates a workflow, not certified APA compliance or inferential validity. If printing the table replaces the chat in Viewer, follow the next page.

6. Recovery, history and export

A table replaced the chat

1. Keep R and the Background Job running. Viewer can continue displaying the table while you inspect it.
2. Within 30 minutes of the last disconnect, open Addins > rcodeagent or use your shortcut. Do not restart the Job through Background Jobs.
3. The same Job restores the last conversation message, attachments, model, effort, synced draft, proposal and target, then scrolls to the end.
4. Auto returns ON or OFF as before if the document passes ID, path and full-content validation. Conflicts or an unverifiable target revoke Auto and show a notice. The captured selection range remains unchanged.

Pending execution authorization is never restored or replayed. Code already submitted may continue running or remain queued in RStudio; returning to chat or closing the assistant does not withdraw it.

Viewer alternates between table and chat. An in-flight request is cancelled; its instruction returns as a draft if no newer draft exists. There is no replay or detached write. Text not yet synced to the server cannot be guaranteed. Close ends the assistant immediately. Restarting R, exceeding the waiting period or starting a new Job loses this temporary recovery.

Local history

Conversations and attachments are stored locally. New conversation starts a separate chat. History lets you resume or delete earlier conversations. When resuming from History, choose the target again and enable Auto only if desired: permissions are not saved. Deleting or exporting a conversation does not sign out of the account.

Export and understand usage

The sidebar Export menu offers HTML, JSON, Markdown and PDF when messages exist. Choose a format and a local destination. Review the output before sharing. Credentials and Auto grants are excluded; conversation text may contain information you supplied. Attachments remain in local history; each exporter uses the validated fields appropriate to its format.

The composer counter shows used/maximum model context. The symbol ≈ marks an estimate; ? means an unknown limit. Provider usage is retained separately when available. Accumulated usage is not a bill and does not necessarily represent your subscription balance or quota.

7. Troubleshooting

Missing addin or unexpected version

Restart RStudio after installation, then check packageVersion("rcodeagent") and About. Inspect .libPaths() if several package libraries exist. Install the correct distribution; do not extract an old archive over current sources.

No models or connection

Use Refresh models. If necessary, Disconnect and Sign in with ChatGPT to repeat interactive authorization. Check internet access, account permissions and proxy/firewall restrictions. Do not expose the local server to the network or publish tokens for diagnosis. rcodeagent_status() reports local status only and is not a connectivity test.

Auto turned off or an edit was rejected

Verify the target and recapture context after inspecting the document. Explicitly choosing selection or file preserves Auto for a valid target; an unrecaptured change, failed capture or conflict may revoke it. For an IPC-channel error, inspect the editor and close/reopen the assistant to start a new Job; never force writes over an uncertain channel. A busy R session can delay editor operations.

Chat does not return after Viewer output

Open the Addin within the waiting period. If you clicked Close, restarted R or waited more than 30 minutes, a new Job starts. Resume saved messages from History and recapture the target. Auto is never recovered from disk.

Rejected attachment, model error or interrupted response

Check extension, UTF-8 encoding and attachment limits. Reduce context if it exceeds the model window. Explicitly select a supported effort if the provider rejects the chosen value; the application does not silently lower it. Review restored drafts before deliberately submitting again. A useful issue report includes package/R/RStudio versions, steps, Auto state and a synthetic example, without personal information.

8. Privacy, limits and references

What you share

Submitting a request sends the instruction, conversation context, editor snapshot and enabled attachments to the service. Local histories contain messages and attachment content. Do not include identifiable participant, patient or student data without appropriate authorization and institutional protections. Prefer synthetic examples or anonymized data.

OAuth credentials are encrypted in local storage; the encryption key is held in the operating system credential store. The chat does not need the tokens. Loopback networking does not protect against other applications running as the same operating-system user. Exports and histories require the same care as your research files.

Scope of this release

Version 0.1.0 incorporates the accepted 0.5.7 implementation and subsequent changes; it replaces the unpublished 1.0.0 designation. Synthetic browser and unit tests do not replace OAuth, inference, refresh and native compatibility checks on each system. User satisfaction is not treated as certification of all these paths. Automatic Environment inspection and general folder access are unavailable. Execution requires an explicit Auto request and affects the main R session; submissions may be queued. Models may make programming, statistical and citation errors.

Supporting documentation

- Posit. RStudio Addins: https://docs.posit.co/ide/user/ide/guide/productivity/add-ins.html
- Posit. Send code to the R console: https://rstudio.github.io/rstudioapi/reference/sendToConsole.html
- r-lib. Install a package from a local file: https://remotes.r-lib.org/reference/install_local.html
- OpenAI. Registration and sign-in: https://developers.openai.com/siwc/token-sharing-open-source/sign-in

Official sources support the Addins, local installation and OAuth mechanisms. rcodeagent-specific behavior is documented from the 0.1.0 sources. Paths and menus may vary across IDE versions. Consult README.md, SECURITY.md and docs/validation/github-readiness.md for the current technical release status; docs/validation/1.0.0.md preserves historical evidence.

Author and maintainer: José Gamarra-Moncayo · gamarramoncayoj@gmail.com